Resoconto · Evoluzione in 9 step dell'assistente di laboratorio

Nov 10, 2026 · @Big Fat Luce

Sintesi

I 9 step sono completati e pubblicati: l'assistente di laboratorio è passato da una CLI con un solo agente a un'applicazione web completa, avviabile con un solo `docker compose up`. Tutto il lavoro è nel repository Esercizio19: `main` e `dev` sono allineati al commit `d04a96a` (11/11/2026).

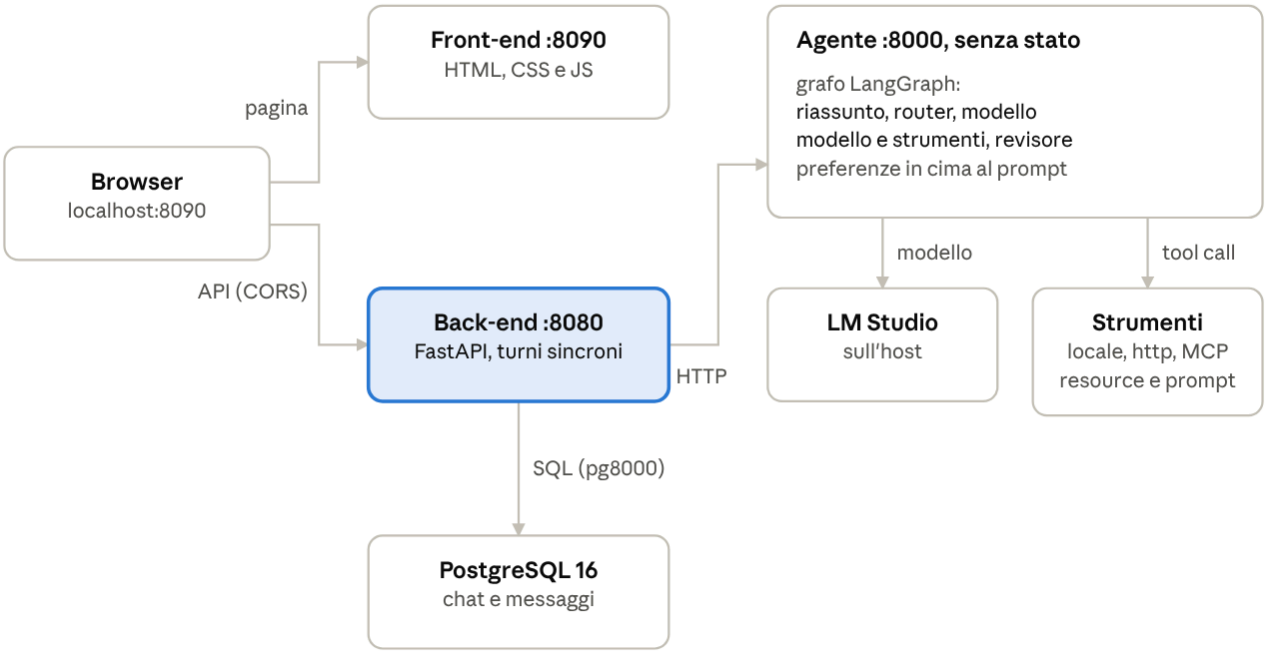
- **Test offline:** 499 passati (erano 334 all'inizio dello step 3), più 6 prove su PostgreSQL vero.
- **Prove reali:** con LM Studio (`google/gemma-4-e2b`) per ogni step, e nel browser (Edge senza finestra) per front-end e Docker Compose.
- **Metodo:** per ogni step specifica, piano e domande approvati prima del codice; un branch `feat/<nome>` da `dev` ; evidenze; commit e merge solo dopo l'ok. Specifiche e piani sono in `docs/superpowers/` .
- **Due fix** fuori dagli step: il revisore che non vedeva strumenti e riassunto, e lo stile delle preferenze che scoraggiava l'uso degli strumenti.

Aggiornamento dell'11/11/2026: il README documenta la scelta sullo streaming (non implementato; percorso previsto con Server-Sent Events, mostrando solo la risposta approvata dal revisore), e le evidenze degli step 8 e 9 (13 screenshot e questo resoconto in PDF) sono nel repository, in `docs/evidenze/` .

Architettura finale

Il browser chiede la pagina al front-end e poi parla solo con il back-end; il back-end salva tutto in PostgreSQL e chiama l'agente, che riceve a ogni turno cronologia, riassunto e preferenze.

Il browser parla solo con il back-end; l'agente non tiene stato



architettura finale · 6 componenti

In Docker Compose solo back-end e front-end sono raggiungibili dall'host (127.0.0.1); agente e PostgreSQL stanno nella rete interna, LM Studio resta sull'host.

Gli step

Ogni step è un branch con il suo commit, unito in `dev` dopo l'approvazione.

Step	Cosa aggiunge	Decisioni chiave	Evidenza principale	Commit
1 Router	rifiuta le domande fuori ambito prima dell'agente	`AMBITO_AGENTE` elenca anche cosa è fuori ambito	«Scrivimi una poesia» → rifiuto fisso, agente non chiamato	17bc21d
2 Revisore	controlla che la risposta copra tutta la domanda, altrimenti l'agente riprova	al massimo `MAX_REVISIONI` tentativi; output non valido = coerente	domanda in due parti completata dopo l'indicazione	34a23c8
3 Riassunto	oltre `SOGLIA_RIASSUNTO` messaggi quelli vecchi diventano un riassunto	taglio solo fra turni; il riassunto inizia con «Utente: …»	«Come mi chiamo?» dopo la compressione → «Ti chiami Prabhjot»	2d10667
4 Preferenze	resource MCP `utente://preferenze` letta una volta a inizio sessione	tre campi validati, in cima al prompt; predefinite con WARNING	«Che ore sono?» → America/New_York senza chiedere il luogo	3cbe075

Step	Cosa aggiunge	Decisioni chiave	Evidenza principale	Commit
5 Prompt	prompt MCP `spiega` e `riassumi_conversazione` con `/prompts` e `/prompt`	controllo degli argomenti nell'host; regola fissa del router per `name="prompt"`	argomento mancante o sconosciuto bloccato senza chiamare il modello	efd45ff
6 Agente HTTP	l'agente diventa un servizio FastAPI senza stato	cronologia solo `{ruolo, testo}`; `name` dal client → 422; contratto `docs/openapi-agente.json`	strumenti spenti 503, modello spento 503 con messaggio diverso	d907495
7 Back-end	conversazioni in PostgreSQL, turni sincroni verso l'agente	pg8000; storico completo e stato dell'agente separati; domanda in errore + `/riprova`	chat presenti dopo il riavvio; agente spento → domanda salvata in errore	e1c9200
8 Front-end	chat web in HTML, CSS e JavaScript senza librerie	solo il back-end, `BACKEND_URL` da `/config.js`, CORS solo per il front-end	attesa con cronometro, «Strumenti usati», «Riprova» riuscito	30c5a4e
9 Docker Compose	applicazione completa con un comando	`0.0.0.0` nei container; `localhost` per browser e CORS; PostgreSQL senza porte	chat presente dopo `down` e `up` (volume `pgdati`)	ca80750

Fix: `cd9a2a2` (il revisore vede strumenti e riassunto) e `5eae0d9` (lo stile sintetico non scoraggia gli strumenti).

Problemi trovati e soluzioni

Ogni problema è stato diagnosticato con il modello vero prima di cambiare il codice.

Problema	Causa	Soluzione
Il nome dell'utente si perdeva dopo la compressione	il riassunto non diceva chi è l'utente	istruzioni del riassunto con la riga «Utente: …» (step 3)
Il revisore bocciava risposte giuste (ora di Tokyo, nome)	vedeva solo domanda e risposta, non strumenti e riassunto	`fix(revisore)`: strumenti del turno e riassunto come dati attendibili

Problema	Causa	Soluzione
Con le preferenze l'agente faceva i calcoli a mente (una volta «96»)	il blocco delle preferenze rende incerto l'uso degli strumenti	`fix(preferenze)`: stile «breve ed essenziale (usa comunque gli strumenti)»; limite annotato
Il router rifiutava `/prompt spiega` (2 volte su 6 accettato)	la regola dello step 1 mette fuori ambito le domande di teoria	regola fissa nel nodo router per i messaggi marcati dal comando `/prompt`
psycopg 3 non si caricava	Windows blocca la sua DLL (controllo delle applicazioni)	driver pg8000, solo Python (scelta dell'utente)
Porta 5432 occupata sull'host	un altro PostgreSQL già attivo	container sulla porta 55432; in Compose nessuna porta pubblicata
Prima domanda di una chat nuova: domanda e cronometro sparivano	creare la chat ridisegnava i messaggi	la chat si crea prima di mostrare l'attesa (step 8)
Fine turno: attesa ferma senza il riquadro d'errore	l'invio si riattivava prima di ricaricare la chat	prima si ricarica, poi si riattiva; file del front-end con `no-cache`

Avvio e verifica

Con LM Studio avviato sull'host e il `.env` compilato (`LMSTUDIO_MODEL`, `AMBITO_AGENTE`, `POSTGRES_PASSWORD`), l'applicazione parte con un comando e la chat si apre su http://localhost:8090.

1. `docker compose up -d --build` avvia postgres, agente, back-end e front-end.
2. `Invoke-RestMethod http://localhost:8080/stato` deve dare database e agente `ok`.
3. Aprire http://localhost:8090 (non 127.0.0.1: il CORS accetta solo `localhost`).
4. `docker compose down` ferma tutto; le chat restano nel volume `pgdati`.

Test (da `assistente-laboratorio/`):

- offline: `.\.venv\Scripts\python.exe -m pytest -v` (499 passati);
- PostgreSQL vero: `... -m pytest -m postgres -v` (6 passati);
- con LM Studio: `... -m pytest -m reale -v` (lente: 13-20 minuti per una prova lunga).

La CLI resta disponibile: `docker compose run --rm assistente`, oppure `python -m assistente` sull'host.

Limiti noti e possibili sviluppi

I limiti dipendono quasi tutti dal modello piccolo (`gemma-4-e2b`): il codice li segnala nei log ma non li elimina.

- **Calcoli a mente:** con le preferenze l'agente usa lo strumento 0-2 volte su 3 nei calcoli facili (3 su 3 senza preferenze); il risultato di solito è giusto ma non è garantito.
- **Router variabile:** in una prova ha classificato «Quanto fa 137 * 49?» come fuori ambito.
- **Revisore e `riassumi_conversazione` :** vede solo gli ultimi 6 messaggi e ha bocciato due volte un riassunto giusto di una sessione più lunga.
- **Lentezza:** un turno con LM Studio dura 1-3 minuti; front-end e proxy devono avere timeout alti (900 s verso l'agente).
- **Docker:** nei container non c'è `preferenze.json` , quindi si usano le preferenze predefinite.

Possibili fix, ciascuno con una sua specifica: più contesto al revisore per `riassumi_conversazione` ; un controllo «calcolo senza strumento» nel revisore; streaming delle risposte al posto dell'attesa sincrona.